



Machine Learning for Petroleum Engineers Using Python

Módulo 3 · Clase 5

SVM, Pipelines y Validación Honesta — Cierre del Módulo



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: la Última Clase del Módulo

✓ Ya saben:

- ✓ Predecir un **número** y una **etiqueta** (C1–C2)
- ✓ Árboles y **Random Forest** (C3)
- ✓ Multiclase, codificar, imputar, GridSearchCV (C4)
- ✓ Métricas + **costo de negocio**, siempre

*Y traemos una deuda anotada desde la Clase 2: validar con **pozos completos por fuera**. Hoy se paga.*

▶▶ Hoy:

- Un problema nuevo: **integridad de ductos** (con fuego incluido)
- El último modelo del módulo: **SVM** y el **kernel trick**
- **Pipelines**: el tubo que ordena todo el flujo
- ★ **Validación honesta** y **SMOTE** para clases raras

*Al final: el marcador y el **mapa de modelos** de todo el módulo.*

Integridad de Ductos

Un problema nuevo: ya no preguntamos qué roca es — preguntamos qué se va a romper

Ustedes Ya Hacen Esto

Piensen en el ingeniero de integridad de cualquier operadora con líneas de flujo.

✂ SU PROBLEMA DIARIO

Tiene **cientos de kilómetros** de línea y presupuesto para inspeccionar **unos pocos**. ¿Cuáles primero? Decide mirando: edad, recubrimiento, presión, historial...

Es decir: decide **comparando con fallas pasadas**.

- Eso es exactamente un problema de **clasificación**: dadas las características de una línea, ¿qué tipo de falla es la más probable?
- Hoy lo haremos con **fallas reales**: cada incidente reportado al regulador de EE. UU. trae la causa investigada **y el costo en dólares**.
- ✓ Dos preguntas de negocio: ¿fue **corrosión**? Y la que quema: ¿qué fugas terminan en **fuego**?

El Dataset: Incidentes Reales, Reportados por Ley

PHMSA (el regulador de ductos de EE. UU.): toda falla significativa se reporta, se investiga y se publica.

- ☰ Incidentes en **gasoductos de transmisión**, 2010–hoy. Nos quedamos con los que ocurrieron **en la tubería** y tienen reporte completo: **638 incidentes** de **132 operadores**.
- 🔍 Cada uno trae: características físicas de la línea, la **causa investigada**, si hubo **ignición**, y el **costo total en dólares**.
- ✅ Datos públicos del gobierno de EE. UU. Curados a un CSV liviano, en español, en el repo del curso.

\$ EL COSTO ES REAL

Incidente mediano por corrosión: **\$325 000**.

Incidente mediano **con ignición**: **\$896 000** — casi 4 veces el de una fuga sin fuego (\$234 000).

El hilo de costo del módulo, por primera vez, **sin supuestos**: dólares reportados.

Acotar el Problema (Regla de la Casa)

Pregunta de negocio	Dada una línea y sus condiciones: ¿la falla fue corrosión ? (y después: ¿la fuga terminó en ignición ?)
Target	CORROSION (sí/no) — binario, como en la Clase 2
Features	6 numéricas (diámetro, edad, presiones, ubicación, costa afuera) + RECUBRIMIENTO (categórica → one-hot, como en la C4)
Métrica de éxito	accuracy y recall de corrosión — y validación honesta (el tema de hoy)
Récord a batir	el modelo tonto (nunca corrosión): 62 %

- ✓ Fíjense: todo lo de la tabla **ya lo saben hacer**. Lo nuevo de hoy es el modelo (SVM), el orden (pipeline) y la honestidad (validación).

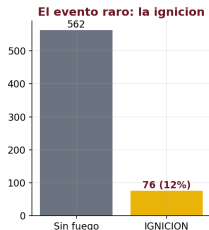
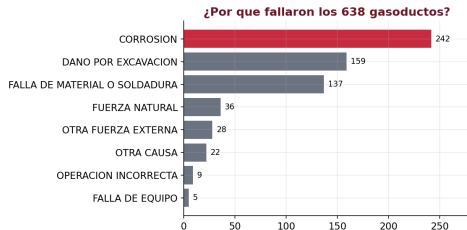
Las Variables, Una a Una

Todas se explican en una línea — ese fue un criterio para elegir este dataset.

Columna	Tipo	Qué es
DIAMETRO_PULG	numérica	diámetro de la línea (pulgadas)
EDAD_ANIOS	numérica	años desde la instalación hasta el incidente
PRESION_MAX_PSI	numérica	presión máxima de operación permitida (MOP)
PRESION_INCIDENTE_PSI	numérica	presión al momento del incidente
RECUBRIMIENTO	categorica	alquitrán, asfalto, cinta fría, sin recubrimiento...
CLASE_UBICACION	numérica (1-4)	densidad poblacional alrededor (1 rural → 4 urbana)
COSTA_AFUERA	binaria	¿la línea es costa afuera? (0/1)
CAUSA / IGNICION	targets	causa investigada; ¿hubo fuego?
COSTO_USD	informativa	costo total reportado del incidente

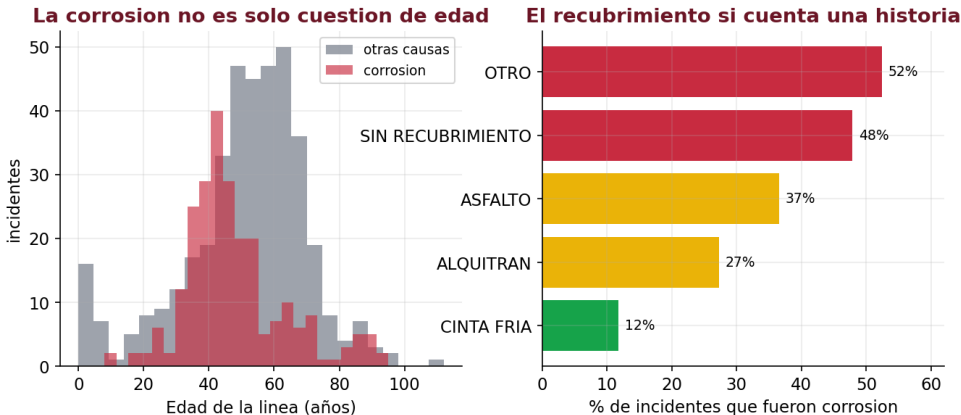
⚠ Hay 2 columnas más en el CSV que **no** están en esta tabla. Son una trampa a propósito — las veremos en un rato.

Exploración: ¿Por Qué Fallan los Gasoductos?



- La **corrosión** es la causa #1: 242 de 638 (**38 %**). Nuestro primer target.
- ⚠ La **ignición** es rara: **12 %**. Guarden ese número: va a romper un modelo más adelante.
- ✓ 38 % vs 62 %:
desbalance **maneable**
(2 a 1, como la Clase 2).

Exploración: la Física Habla



→ Sorpresa: la corrosión **no** es simplemente “línea vieja” — su pico está en los 40 años, y hay líneas de 60+ que fallan por otras causas.

Las Dos Columnas Trampa (Anótenlas)

El CSV trae dos columnas que se conocen después de investigar la falla.

⚠ LAS TRAMPAS

GAS_LIBERADO_MPCS: cuánto gas escapó — se mide **después** de la fuga.

IND_CORROSION_GALVANICA: lo marca el **investigador** cuando ya determinó que hubo corrosión.

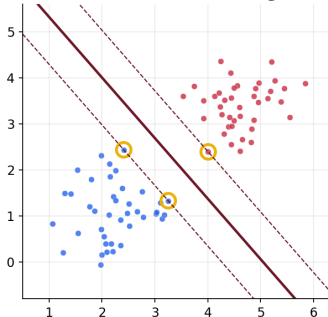
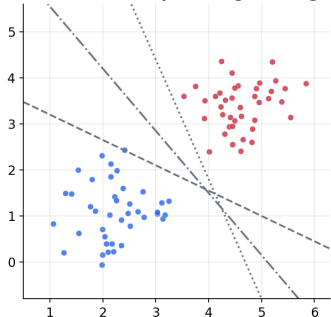
- ? Si nuestro objetivo es **priorizar inspecciones antes de que la línea falle...** ¿podemos usar columnas que solo existen **después** de la falla?
- ⊘ No. Sería **predecir el pasado**. Se llama **fuga de información** y es el error más caro del ML industrial.
- ✓ Regla de oro: solo features que se conocían **ANTES** del evento. En un rato lo mediremos en vivo.

SVM: la Carretera Más Ancha

El último modelo del módulo

Separar Dos Poblados con una Carretera

Muchas rectas separan... ¿cual elegir? SVM: la carretera mas ancha (margen maximo)



- Muchas rectas separan los dos grupos (izquierda). La SVM elige la que deja **la carretera más ancha** posible entre los dos poblados (derecha).
- Las casas que **tocan la cerca** (círculos amarillos) son los **vectores de soporte**: ellas solas definen la carretera. Las demás podrían desaparecer y nada cambia.
- ✓ ¿Por qué la más ancha? Porque un dato nuevo, ligeramente distinto, **sigue cayendo del lado correcto**: margen = seguridad.

SVM en Código: el Primer Intento

El patrón de siempre: importar, crear, entrenar, evaluar. Nada nuevo.

Python

```
from sklearn.svm import SVC

svm = SVC()                # el modelo SVM
svm.fit(X_tr, y_tr)        # aprende del
    ↪ train
print(svm.score(X_te, y_te)) # acierto
    ↪ en test
```

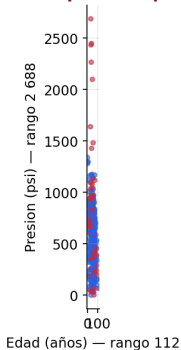
>_ Output

0.776

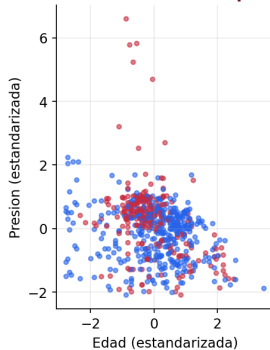
- ✓ **0.776**: ya le gana al tonto (0.62). Cuatro líneas de código.
- ❓ Pero... este modelo mide **distancias** entre incidentes (¿qué tan parecidas son dos líneas?).
- ⚠ Y nuestras variables tienen **escalas muy distintas**. ¿Cuánto nos está costando eso? Siguiendo slide.

El Problema Escondido: las Escalas

SIN escalar: la presión aplasta todo



Escalado: cada variable opina igual



- La presión se mueve en un rango de **3 205 psi**; la edad, en **112 años**; costa afuera, en **1**. Para una distancia, la presión **grita** y las demás susurran.
- Es “el paso parejo” de la Clase 1 otra vez: estandarizar pone a **todas las variables a opinar con la misma voz**.
- ✓ La logística (C2) ya lo necesitaba, los árboles (C3) no, la SVM lo necesita a gritos.

Pipelines: el Tubo

Escalar bien, sin poder equivocarse

Escalar a Mano: 4 Pasos, 2 Errores Clásicos

Así lo hicimos en la Clase 2. Funciona... si nadie se equivoca nunca.

1. Ajustar el escalador **solo con el train**
2. Transformar el train
3. Transformar el test **con el mismo escalador**
4. Entrenar y evaluar con las versiones escaladas

*Cuatro objetos sueltos (escalador, datos crudos, datos escalados, modelo) que hay que coordinar **a mano**, siempre, sin fallar.*

✗ ERROR CLÁSICO 1

Evaluar el test **sin escalar** (o escalado con otro escalador): números basura sin aviso de error.

✗ ERROR CLÁSICO 2

Ajustar el escalador con **todos** los datos antes de separar train/test: el test se **filtra** en el entrenamiento (fuga, otra vez).

¿Qué es un Pipeline? El Tren de Tratamiento

En sus facilidades el crudo entra crudo y sale listo: **separador** → **deshidratador** → **bomba**. Nadie carga baldes entre equipos.



- Cada pieza hace **su** paso y le entrega a la siguiente, **en orden, siempre igual**.
- ✓ Al hacer `.fit()`, el tubo entrena el escalador **y** el modelo, con el train. Al hacer `.predict()`, escala y clasifica. **Imposible olvidar un paso.**

El Pipeline en Código: 5 Puntos Gratis

El mismo SVM de hace 3 slides — ahora con el escalado dentro del tubo.

Python

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import
    StandardScaler

tubo = make_pipeline(
    StandardScaler(),      # paso 1: escalar
    SVC())                 # paso 2: clasificar
tubo.fit(X_tr, y_tr)      # el tubo entero
    ↪ aprende
print(tubo.score(X_te, y_te))
```

>_ Output

0.823

- ✓ De 0.776 a **0.823**: casi **5 puntos gratis** por escalar bien. Sin cambiar el modelo.
- ✓ Y el código quedó **más corto** que la versión manual de 4 pasos.
- ➔ `make_pipeline` arma el tubo con las piezas **en el orden** en que se las pasan.

Los 3 Porqués del Pipeline

De aquí en adelante, en este curso, todo modelo que necesite preparación va en pipeline.

	Porqué	Qué significa
✂	Código más simple	un solo objeto con <code>.fit()</code> / <code>.predict()</code> ; menos variables sueltas, menos renglones
✓	Cero olvidos	el test siempre pasa por el mismo tratamiento que el train — los 2 errores clásicos se vuelven imposibles
⚖	Validación honesta	cuando hagamos validación cruzada (ya viene), el escalador se reajusta dentro de cada pliegue — sin fugas. A mano, casi nadie lo hace bien

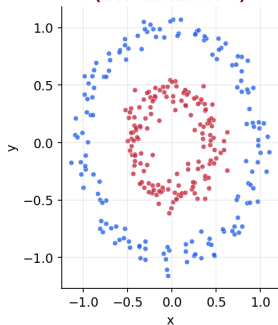
*El tercero es el importante: el pipeline no es comodidad, es **honestidad automatizada**.*

El Kernel Trick

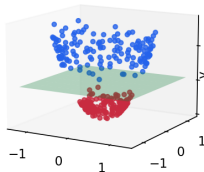
Subir de dimensión para separar lo inseparable

Cuando Ninguna Recta Alcanza

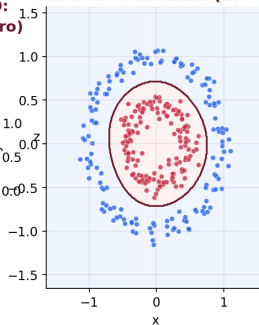
2D: ninguna recta puede
(SVM lineal: 58%)



Dimension nueva A MANO:
 $z = x^2 + y^2$ (distancia al centro)



Kernel rbf: mismo efecto,
sin crear la columna (100%)



- Izquierda: un anillo rodea al centro. La mejor recta se queda en **58 %** — apenas una moneda al aire.
- ✓ Centro: inventamos **a mano** una columna nueva: $z = x^2 + y^2$ (la **distancia al centro**). Vista en 3D, la nube se separa en dos pisos... y un simple **plano** basta.
- ★ Derecha: el kernel rbf logra **exactamente el mismo efecto** — **100 %** — sin que nadie creara la columna. Ese es el truco.

La Idea en Palabras (Sin Matemática)

Si el problema no se puede separar en el plano... míralo desde otra dimensión.

≡ SUBIR DE DIMENSIÓN

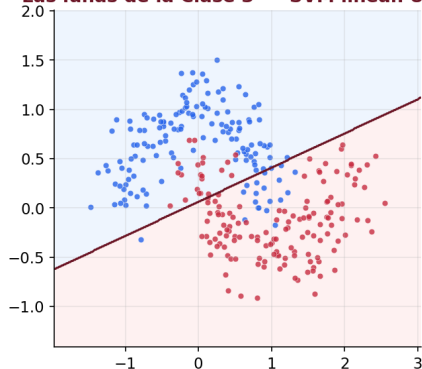
En 2D el anillo es inseparable. Pero al agregar “distancia al centro”, los puntos rojos quedan **abajo** y los azules **arriba**: un plano los separa.

Crear la columna correcta **convierte un problema curvo en uno recto**.

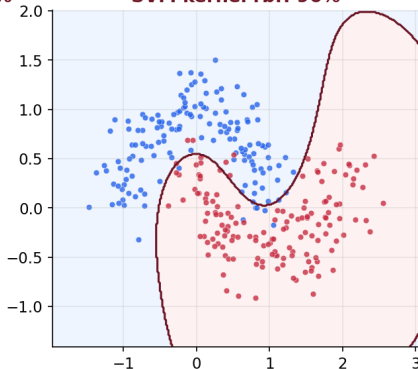
- ⚠ El problema: ¿cuál columna inventar? ¿ x^2 ? ¿ $x \cdot y$? ¿ $\sqrt{x}$? Con nuestras **11 features**, solo los cuadrados y productos de pares ya son **más de 70 columnas** nuevas. Y quizá se necesiten cientos.
- ⚡ **El kernel trick**: una propiedad matemática de la SVM le permite obtener **el efecto** de todas esas columnas **sin calcular ninguna**.
- ✓ En el código, todo el truco es un argumento: `SVC(kernel='rbf')`. Y es el valor por defecto.

Las Lunas Vuelven (Por Última Vez)

Las lunas de la Clase 3 --- SVM lineal: 86%



SVM kernel rbf: 96%

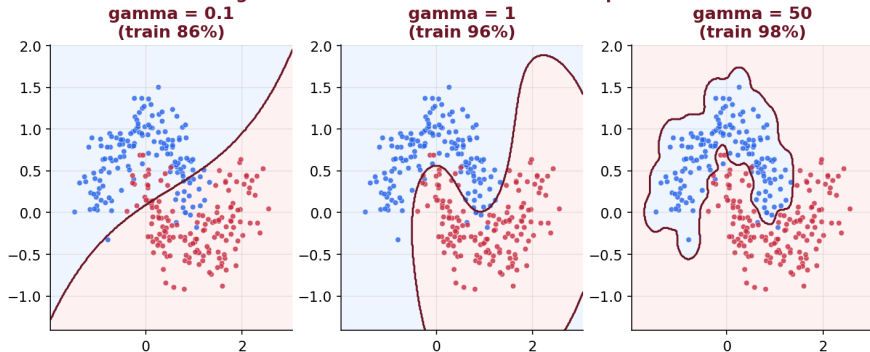


→ En la Clase 3 las lunas derrotaron a la **logística** y las salvó el **árbol**. Hoy, la SVM lineal se queda en 86 %...

✓ ...y el kernel rbf dibuja una frontera **curva y suave**: 96 %. Comparen con los escalones rectos del árbol: dos formas distintas de ser no lineal.

La Perilla gamma: el Radio de Influencia

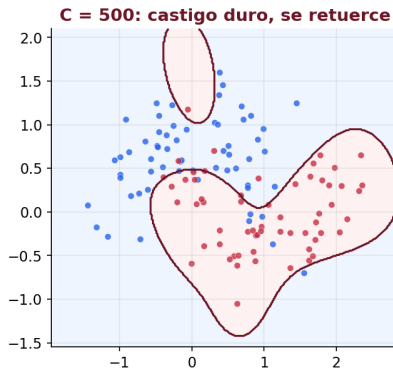
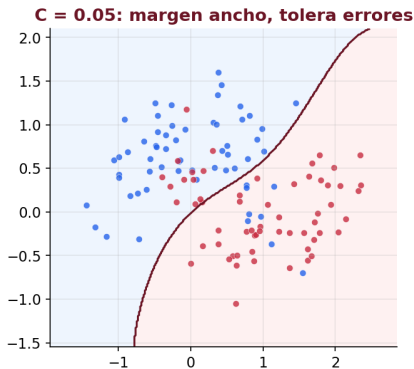
gamma: el radio de influencia de cada punto



→ gamma pequeño = cada punto influye **lejos** → frontera suave, casi recta. gamma grande = cada punto solo manda **en su vecindario** → la frontera se vuelve paranoica y **encierra puntos sueltos**.

⚠ ¿Train 98 % con gamma = 50? Eso ya lo vieron en la C3: huele a **memorizar**, no a aprender. Es el overfitting con otra perilla.

La Perilla C: ¿Cuánto Castigo por Invadir?



→ C chico: carretera **ancha y tranquila** — tolera que algunas casas queden del lado equivocado. C grande: **castigo duro** por cada invasor — la carretera se retuerce para no dejar ninguno.

≡ C y gamma son los **hiperparámetros** de la SVM (¿recuerdan la C4? las perillas que ustedes eligen). ¿Y cómo se eligen **sin hacer trampa**? Sección siguiente.

¿Y en Nuestros Ductos? La Lección Honesta

Probamos las dos SVM en el problema de corrosión. Miren:

Modelo (siempre en pipeline)	Accuracy	F1 corrosión
SVM lineal	0.828	0.732
SVM kernel rbf	0.823	0.730

- El kernel **no ganó**: este problema resultó **casi lineal**. Pasa muchísimo con datos tabulares reales.
- ✓ La lección profesional: **prueben siempre el lineal primero**. Es más rápido, más explicable, y a veces empata o gana. El kernel es un **as bajo la manga** (círculos, lunas), no un reflejo automático.
- ⊙ Esto también es andragogía de ingeniero: nadie perfora con la broca más cara **si la formación no lo pide**.

Afinar sin Hacer Trampa

Validación cruzada, GridSearch y la fuga

Una Foto Puede Mentir: 5 Fotos Mienten Menos

Nuestro split train/test es UNA repartición al azar. ¿Y si nos tocó una fácil? ¿O una difícil?

Python

```
from sklearn.model_selection import  
    ↪ cross_val_score  
  
notas = cross_val_score(tubo, X, y, cv=5)  
print(notas.round(3))    # 5 exámenes  
    ↪ distintos  
print(notas.mean().round(3))
```

>_ Output

```
[0.789 0.781 0.812 0.787 0.764]  
0.787
```

- ↻ cv=5: parte los datos en **5 pliegues**; entrena 5 veces, cada vez dejando un pliegue distinto como examen. **Todos los datos examinan una vez.**
- 👁 Miren la dispersión: de **0.764 a 0.812**. Una sola foto pudo decirnos cualquiera de esos números.
- ✓ Reportar **media de 5** > reportar 1 número con suerte. Esto es lo que GridSearchCV hacía por dentro en la C4.

GridSearch Sobre el Tubo Completo

Novedad de hoy: la parrilla nombra la pieza del tubo — `svc__C` = “la perilla C, de la pieza `svc`”.

Python

```
from sklearn.model_selection import
    GridSearchCV

perillas = {'svc__C': [0.1, 1, 10, 100],
            'svc__gamma': [0.01, 0.1, 1]}
buscador = GridSearchCV(tubo, perillas,
    cv=5)
buscador.fit(X_tr, y_tr)    # 12 combos x 5
    pliegues
print('mejor:', buscador.best_params_)
```

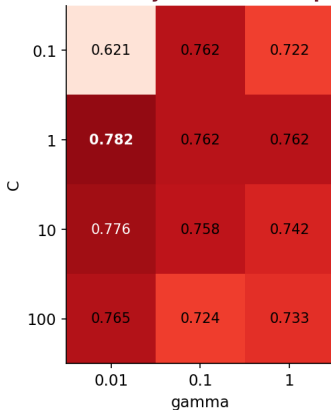
>_ Output

```
mejor: {'svc__C': 1, 'svc__gamma': 0.01}
```

- ✓ Como el tubo va **completo** dentro de la búsqueda, el escalador se reajusta **en cada pliegue**: cero fugas. (El porqué #3 del pipeline, cumplido.)
- ➔ Con las perillas ganadoras, el test da **0.828** (veníamos de 0.823): mejora fina, sin trampa.
- 👁 gamma ganador = **0.01** (el más suave de la parrilla): otra pista de que el problema es casi lineal.

El Mapa de Perillas Completo

GridSearchCV: accuracy de validacion por combinacion



- Cada casilla = una combinación de perillas, calificada con sus **5 exámenes internos**.
- ✓ La zona buena es **amplia y estable** (0.76–0.78): el modelo no está colgado de una perilla mágica. Eso da confianza.
- ⚠ La esquina $C = 0.1$ con $\gamma = 0.01$ se desploma (0.62 = el tonto): castigo tan suave que el modelo **se rinde** y dice siempre “no corrosión”.

La Trampa, Medida en Vivo

¿Recuerdan las 2 columnas prohibidas? Agreguemos UNA al modelo, “por curiosidad”.

Features	Accuracy test
Las 11 honestas (lo que sabías ANTES de la falla)	0.823
+ IND_CORROSION_GALVANICA (del reporte del investigador)	0.906

- ⚠ ¡8 puntos gratis! Pero es un modelo que necesita... **que la falla ya haya ocurrido y esté investigada.** Inútil para priorizar inspecciones. **Predice el pasado.**
- ⊘ Así se ve la **fuga de información** en la vida real: un número espectacular que se derrumba en producción. Si un resultado parece demasiado bueno, **pregunten qué sabía el modelo y cuándo.**
- ✓ Ya la vimos 3 veces en el módulo: escalar antes del split (hoy), imputar con todo el dataset (C4), y esta. **Misma enfermedad, tres disfraces.**

La Pregunta Final: ¿el Modelo Viaja?

Nuestros pliegues mezclan operadores. Pero el negocio pregunta: ¿funciona con un operador **NUEVO**?

Python

```
from sklearn.model_selection import GroupKFold

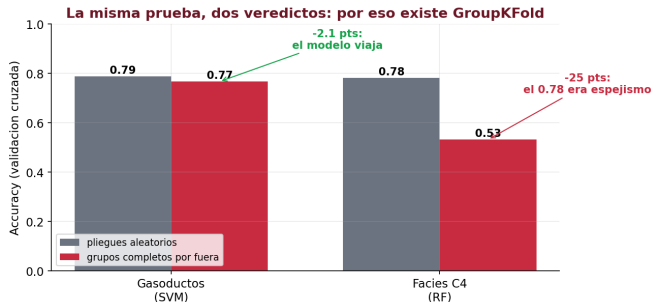
notas = cross_val_score(
    tubo, X, y,
    cv=GroupKFold(5),           # pliegues por
    ↪ grupo                     # el grupo:
    groups=d['OPERADOR_ID'])   ↪ operador
print(notas.mean().round(3))
```

>_ Output

0.766

- 👤 GroupKFold: cada pliegue deja **operadores completos** por fuera. El examen es con empresas que el modelo **jamás vio**.
- ✓ 0.787 → 0.766: cae solo **2 puntos**. Buena noticia: el modelo aprendió física de ductos, no mañas de cada operador. **Viaja**.
- ❓ ¿Y si hubiéramos hecho esto con las **facies** de la Clase 4, dejando **pozos** por fuera?

La Deuda del Módulo, Pagada



⚠ Las facies: aleatorio **0.78** → pozos por fuera **0.53. 25 puntos de espejismo:** medios pies vecinos del mismo pozo estaban repartidos entre train y test.

➔ Esta es la deuda que anotamos en las Clases 2, 3 y 4. El 0.76 de facies era real **solo para pozos ya vistos.**

✓ GroupKFold no “mejora” el modelo: **les dice la verdad.** A veces certifica (ductos), a veces desinfla (facies).

Ambas valen oro.

Su Turno: Afinar con Honestidad

🕒 15 min

Todo con pipeline. Todo sin tocar el test hasta el final.

1. Armen un pipeline con `StandardScaler` y `SVC(kernel='linear')` y calculen su `cross_val_score` con `cv=5`. ¿Confirma que el lineal compite con el rbf?
2. Amplíen la parrilla del `GridSearchCV` agregando `'svc__kernel': ['linear', 'rbf']`. ¿Quién gana ahora y con qué perillas?
3. Evalúen su mejor modelo con `GroupKFold` por operador. ¿Cuánto cae respecto a los pliegues aleatorios?
4. Pregunta de negocio: escriban **una frase** para el gerente de integridad diciendo qué accuracy debe esperar **y por qué esa** (¿la de pliegues aleatorios o la de operadores por fuera?).

SMOTE: Fabricar lo Raro

La segunda pregunta: ¿qué fugas terminan en fuego?

La Segunda Pregunta de Negocio: el Fuego

Mismo dataset, nuevo target: **IGNICION**. Solo el 12 % de las fugas se encienden.

- \$ ¿Por qué importa? El incidente mediano **con fuego** cuesta **\$896 000**; sin fuego, \$234 000. Casi **4 veces más** — sin contar personas.
- Si pudiéramos anticipar **qué líneas, al fallar, tienden a encenderse**, priorizaríamos válvulas de cierre automático, espaciamiento, planes de emergencia.
- ⚠ Pero la clase que importa es **rara**: 76 fuegos contra 562 fugas tranquilas.

? PREGUNTA ANTES DE MODELAR

El modelo tonto (“nunca hay fuego”) acierta el **88 %**.

¿Recuerdan al **modelo perezoso** de la C4, ciego a la dolomita? ¿Qué creen que hará nuestro SVM con los fuegos?

El Modelo Ciego: Accuracy Alta, Cero Fuegos

Entrenamos el mismo tubo de siempre con el nuevo target. Miren las dos métricas juntas:

Python

```
from sklearn.metrics import recall_score

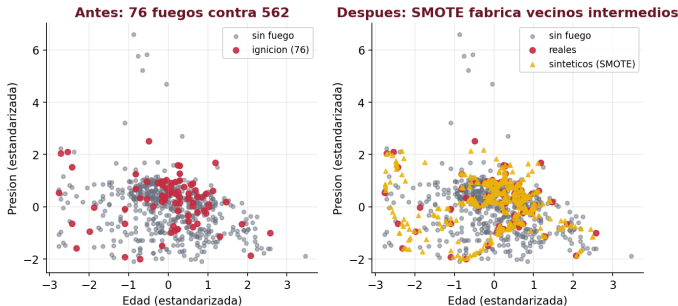
tubo.fit(X_tr, y_fuego_tr)  # mismo tubo,
    ↪ target fuego
pred = tubo.predict(X_te)
print(tubo.score(X_te, y_fuego_te))  #
    ↪ accuracy
print(recall_score(y_fuego_te, pred)) # y el
    ↪ recall...
```

>_ Output

```
0.88
0.0
```

- ✗ Accuracy **0.88...** y recall **0.000**: de los 23 fuegos del test, detectó **cero**.
- ⚠ Con 12 % de fuegos, ignorarlos por completo **ya da** 88 %. La accuracy **premia la ceguera** — el modelo tonto de la C2 y el perezoso de la C4, reunidos.
- ❓ El modelo casi no ve fuegos al entrenar (53 contra 393). ¿Y si le **fabricamos** más ejemplos?

SMOTE: Fabricar Casos Parecidos a los Raros



- SMOTE toma cada fuego real, busca su fuego **más parecido** y crea puntos **intermedios** (triángulos): sintéticos pero **creíbles**.
- ✓ El train pasa de 53 fuegos a **393 contra 393**. Regla sagrada: **SOLO al train** — inventar datos en el examen sería **otra fuga**.

SMOTE en Código: una Pieza Más en el Tubo

La librería `imblearn` trae un pipeline que sabe aplicar SMOTE solo donde toca.

Python

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import make_pipeline

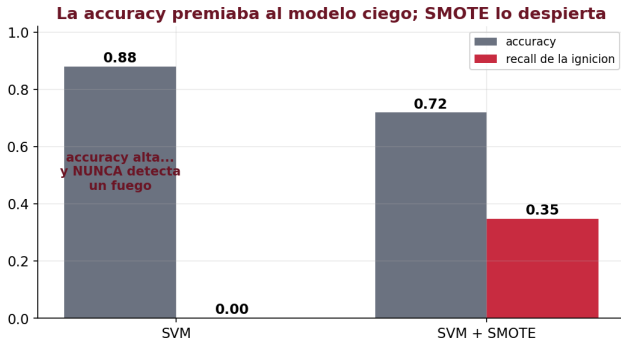
tubo_smote = make_pipeline(
    StandardScaler(),          # escalar
    SMOTE(random_state=42),    # equilibrar (solo
    ↪ train)
    SVC())                    # clasificar
tubo_smote.fit(X_tr, y_fuego_tr)
```

>_ Output

```
Pipeline(steps=[('standardscaler', ...),
                 ('smote', ...), ('svc', ...)])
```

- ✓ El tubo aplica SMOTE al entrenar... y lo **omite** al predecir. Exactamente la regla sagrada, automatizada.
- ➔ !pip install imbalanced-learn en Colab si hace falta (como vimos en la Clase 3).
- 💡 Fíjense: el pipeline dejó de ser un truco de comodidad y se volvió **la arquitectura** donde todo encaja.

El Despertar del Modelo (y su Precio)



✓ Recall del fuego: **0.00** → **0.35**. Ahora detecta 8 de 23 fuegos del test.

⚠ El precio: 39 **falsas alarmas** (accuracy baja a 0.72; 1 de cada 5-6 alertas es fuego real).

⚖ ¿Vale la pena? Con fuego a **\$896 000** y falsa alarma al costo de una **revisión preventiva**, la cuenta sale sola. Es el trade-off del **umbral** de la C2, con otra herramienta.

Su Turno: Domar el Fuego

🕒 15 min

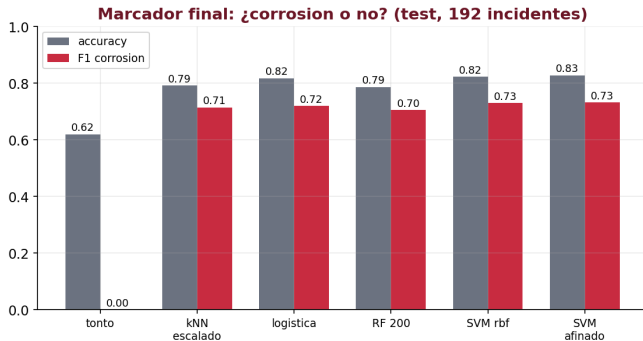
El SVM despertó pero paga caro. ¿Pueden conseguir un mejor trato?

1. Armen `RandomForestClassifier` + SMOTE (con el pipeline de `imblearn`, sin escalador: los árboles no lo necesitan). Comparen `accuracy` y `recall` contra el SVM + SMOTE.
2. Prueben `SMOTE(sampling_strategy=0.5)` — fabricar fuegos hasta la **mitad** de la clase grande, no hasta igualar. ¿Mejora el `recall`/falsas alarmas?
3. Saquen la `confusion_matrix` de su mejor opción y cuenten: ¿cuántos fuegos detecta? ¿cuántas falsas alarmas genera?
4. Pregunta de negocio: con fuego = \$896 000 y revisión preventiva = \$20 000, escriban **una frase** para el gerente HSE recomendando (o no) desplegar su modelo.

Cierre del Módulo 3

Cinco clases, un método

El Marcador Final de la Clase



➔ Todos los caminos llegaron a **0.79–0.83**: cuando la señal es la que es, ningún modelo hace magia.

✓ Lo que sí cambió el juego: **escalar** (+5), **validar honesto** (saber que el número es real) y **SMOTE** (ver lo raro).

🎯 El método pesa más que el modelo. Ese es el cierre del módulo.

El Mapa de Modelos del Módulo:Cuál Usar Cuándo

Modelo	Su fuerte	Su debilidad	¿Escalar?
Regresión lineal	explicable, rápida, un número por variable	solo relaciones de paso parejo	sí
Logística	igual, pero para clasificar; da probabilidades	fronteras rectas	sí
Árbol	se dibuja y se explica solo	inestable, memoriza fácil	no
Random Forest	potente sin afinar casi nada	caja más oscura, pesado	no
SVM	margen máximo; kernel para lo curvo	lenta con muchos datos; C y gamma	sí, siempre

- ➔ Receta profesional: empiecen por el **simple** (logística / lineal), luego **Random Forest**, y SVM cuando el problema lo pida. Comparen **siempre** contra el tonto.
- Para su radar (no lo vimos): *gradient boosting* (XGBoost) — el primo competitivo del bosque — y *k-NN*, el clasificador por vecinos. Con lo de hoy pueden aprenderlos solos.

Lo Que Aprendimos Hoy

Conceptos

- ✓ SVM: la carretera más ancha (margen, vectores de soporte)
- ✓ Kernel trick: subir de dimensión sin pagar el costo
- ✓ Pipeline: honestidad automatizada
- ✓ Validación cruzada: 5 fotos mienten menos
- ✓ Fuga de información: sus 3 disfraces
- ✓ GroupKFold: ¿el modelo viaja? (facies: no viajaba)
- ✓ SMOTE: fabricar lo raro, solo en el train

Herramientas

- ✗ SVC (kernel, C, gamma)
- ✗ make_pipeline, StandardScaler
- ✗ cross_val_score, GroupKFold
- ✗ GridSearchCV con svc__C
- ✗ SMOTE + pipeline de imblearn

El número del día:

facies con pozos por fuera: **0.78** → **0.53**. El espejismo medía **25 puntos** — y solo la validación honesta lo vio.

El Arco Completo del Módulo 3

Cinco clases, y en cada una la misma pregunta: ¿qué decisión habilita este modelo y cuánto cuesta equivocarse?

- C1** Predecir un **número** (regresión) — el sónico de Volve, R^2 , residuos
- C2** Predecir una **etiqueta** (logística) — litología, métricas, **costo de negocio** y umbral
- C3** Romper la recta (**árboles y bosque**) — las lunas, overfitting, el récord de costo
- C4** Elegir entre **muchas** (multiclase) — facies, codificar, imputar, macro/weighted, penalización
- C5** Hacerlo **honesto** (SVM, pipeline, CV, GroupKFold, SMOTE) — ductos, fuego, y la verdad sobre el 0.78

*Ya no son personas que corren `.fit()`: son ingenieros que saben **cuándo crearle a un modelo**.*

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Fin del Módulo 3 · SLB Ecuador · UDLA · 2026